Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.: 352

Name: Gohel Smit

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

In [5]:
```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

class Area:
    def calculate_area(self, rect):  # object as argument
        return rect.length * rect.width

r1 = Rectangle(10, 5)

a = Area()
result = a.calculate_area(r1)

print(f"Area of Rectangle: {result}")
```

```
Area of Rectangle: 50
```

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

In [6]:
```
class Square:
    def __init__(self, side):
```

```
        self.side = side

    def area(self):
        result = self.side * self.side
        self.output(result)  # calling output() from area()

    def output(self, result):
        print(f"Side: {self.side}")
        print(f"Area of Square: {result}")

s1 = Square(5)
s1.area()
```

```
Side: 5
Area of Square: 25
```

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

In [9]:

```
class Rectangle:
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        result = self.length * self.width
        self.output(result)

    def output(self, result):
        print(f'Length: {self.length}')
        print(f'Width: {self.width}')
        print(f"Area of Rectangle: {result}")

    @classmethod
    def check_sides(cls, length, width):
        if length == width:
            print("THIS IS SQUARE")
            return False
        return True

l = 4; w = 6
if Rectangle.check_sides(l, w):
    r1 = Rectangle(l, w)
    r1.area()
```

```
Length: 4
Width: 6
Area of Rectangle: 24
```

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to accees the private attribute from outside of the class.

In [10]:
```
class Square:
    def __init__(self, side):
        self.__side = side  # private attribute

    def get_side(self):  # Getter method
        return self.__side

    def set_side(self, side):  # Setter method
        if side > 0:
            self.__side = side
        else:
            print("Invalid side value")

s1 = Square(5)
print(f"Side: {s1.get_side()}")
s1.set_side(10)
print(f"Updated Side: {s1.get_side()}")


# name mangling   __var  →  _ClassName__var
```

```
Side: 5
Updated Side: 10
```

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balanace. It has two methods getBalance() and printBalance().

In [2]:
```
class Profit:
    def get_profit(self):
        self.profit = float(input("Enter profit: "))

class Loss:
    def get_loss(self):
        self.loss = float(input("Enter loss: "))

class Balance_sheet(Profit, Loss):
    def get_balance(self):
        self.balance = self.profit - self.loss

    def print_balance(self):
        print(f"Profit: {self.profit}")
        print(f"Loss: {self.loss}")
        print(f"Balance: {self.balance}")
```

```
b1 = Balance_sheet()
b1.get_profit()
b1.get_loss()
b1.get_balance()
b1.print_balance()
```

```
Profit: 7.0
Loss: 0.7
Balance: 6.3
```

15) WAP to demonstrate all types of inheritance.

Single Inheritance

In [5]:
```
class Parent:
    def show(self):
        print("Parent class")

class Child(Parent):
    def display(self):
        print("Child class")

c = Child()
c.show()
c.display()
```

```
Parent class
Child class
```

Multiple Inheritance

In [6]:
```
class A:
    def showA(self):
        print("Class A")

class B:
    def showB(self):
        print("Class B")

class C(A, B):
    def showC(self):
        print("Class C")

c = C()
c.showA()
c.showB()
c.showC()
```

```
Class A
Class B
Class C
```

Multilevel Inheritance

In [12]:
```
class Grand_parent:
    def gp_show(self):
```

```
        print("Grandparent class")

class Parent(Grand_parent):
    def p_show(self):
        print("Parent class")

class Child(Parent):
    def c_show(self):
        super().gp_show()
        super().p_show()
        print("Child class")

c = Child()
c.gp_show()
c.p_show()
c.c_show()
```

```
Grandparent class
Parent class
Grandparent class
Parent class
Child class
```

Hierarchical Inheritance

In [13]:

```
class Parent:
    def show(self):
        print("Parent class")

class Child1(Parent):
    def display1(self):
        print("Child1")

class Child2(Parent):
    def display2(self):
        print("Child2")

c1 = Child1()
c2 = Child2()

c1.show()
c1.display1()

c2.show()
c2.display2()
```

```
Parent class
Child1
Parent class
Child2
```

Hybrid Inheritance

In [14]:

```
class A:
    def showA(self):
        print("Class A")

class B(A):
    def showB(self):
        print("Class B")
```

```python
class C(A):
    def showC(self):
        print("Class C")

class D(B, C):
    def showD(self):
        print("Class D")

d = D()
d.showA()
d.showB()
d.showC()
d.showD()
```

```
Class A
Class B
Class C
Class D
```

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the super() and then initialize the salary attribute.

In [15]:
```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

class Employee(Person):
    def __init__(self, name, age, salary):
        super().__init__(name, age)
        self.salary = salary

    def display(self):
        print(f"Name: {self.name}")
        print(f"Age: {self.age}")
        print(f"Salary: {self.salary}")

e1 = Employee("Smit", 1, 70000000)
e1.display()
```

```
Name: Smit
Age: 1
Salary: 70000000
```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [18]:
```
class Shape:
    def draw(self):
        pass

class Rectangle(Shape):
    def draw(self):
        print("Drawing Rectange")

class Circle(Shape):
    def draw(self):
        print("Drawing Circle")

class Triangle(Shape):
    def draw(self):
        print("Drawing Rectange")

shapes = [Rectangle(), Circle(), Triangle()]

for shape in shapes:
    shape.draw()  # Rectangle().draw(), Circle().draw(), Triangle().draw()


# def draw_shape(shape):
#     shape.draw()

# draw_shape(Rectangle())
# draw_shape(Circle())
# draw_shape(Triangle())
```

```
Drawing Rectange
Drawing Circle
Drawing Rectange
```